

## Document Summary

*This note covers the foundational concepts of probability and statistics essential for advanced machine learning, including sample spaces, random variables, probability distributions, inference techniques like Bayes' rule and Naïve Bayes classification, and estimation methods such as MLE and MAP. It explores uncertainty modeling through axioms, independence, and continuous distributions, with practical examples in diagnosis, anomaly detection, and recommender systems. Key statistical measures like bias, variance, and MSE are discussed to evaluate model performance under i.i.d. assumptions.*

# Advanced Machine Learning: Fundamentals of Probability and Statistics

## Why Basic Statistics in Machine Learning

**Machine learning** often requires making predictions in situations filled with uncertainty. To handle this uncertainty effectively during data analysis, **probability** and **statistics** offer essential tools that quantify and manage it.

### Practical Applications

- **Diagnosis:** For instance, predict the probability that a patient will suffer a heart attack in the next year, based on their clinical history. This involves assessing risks from incomplete or noisy medical data.
- **Anomaly detection:** Evaluate how likely a set of readings from an airplane's jet engine is under normal operating conditions. Unusual patterns might signal a failure, helping detect anomalies early.
- **Reinforcement learning:** An agent must act intelligently in an environment, taking into account the probability of receiving a high reward for each possible action. This guides decision-making in uncertain scenarios like games or robotics.
- **Recommender systems:** For a large online bookseller, estimate the probability that a specific user will buy a particular book. This powers personalized recommendations by modeling user preferences amid vast choices.

The world is inherently uncertain for several reasons:

- **Uncertain inputs:** This includes missing data or noisy data, where measurements are imprecise due to sensors or human error.
- **Uncertain knowledge:** Multiple causes can lead to multiple effects; there might be an incomplete list of conditions or effects; causality may not be fully understood; or outcomes could be stochastic (random by nature).
- **Uncertain outputs:** Induction, or generalizing from examples, is always uncertain; even incomplete deductive inference can introduce doubt.

Probability provides a mathematical way to summarize and work with uncertainty arising from all these sources. By assigning numerical values to possibilities, it enables reasoned predictions. See [Machine Learning](#) for broader context.

## Sample Spaces and Events

### *Sample Space*

A **sample space**  $\Omega$  represents the set of all possible outcomes for a random experiment, which could be conceptual (like a thought experiment) or physical (like a real-world measurement). The sample space  $\Omega$  can be finite, with a limited number of outcomes, or infinite, extending without bound.

### *Sample Space Examples*

- Rolling a fair six-sided die:  $\Omega = \{1, 2, 3, 4, 5, 6\}$ .
- Flipping a single coin:  $\Omega = \{H, T\}$ , where  $H$  is heads and  $T$  is tails.
- Flipping a coin three times:  $\Omega = \{HHH, HHT, HTH, HTT, THH, THT, TTH, TTT\}$ , capturing all sequences.
- A person's age:  $\Omega$  consists of all positive integers  $(1, 2, 3, \dots)$ , which is countably infinite.
- A person's height:  $\Omega$  is the set of all positive real numbers, which is uncountably infinite.

### *Event*

An **event** is simply a subset of the sample space  $\Omega$ , defining a collection of outcomes that share a common property.

### Event Examples

- In a book, the event that it is open at an odd-numbered page.
- For rolling a die, the event that the output number is less than 4:  $\{1, 2, 3\}$ .
- For a random person's height  $A$ , the event that their height falls between  $x$  and  $y$ :  $\{A \mid x < A < y\}$ .

The core question in probability is: What is the probability of a particular event occurring? This leads us to formal definitions and rules.

## Axioms of Probability

### Probability

The **probability**  $P(A)$  of an event (which is a subset)  $A$  is a function that maps  $A$  to a value in the interval  $[0, 1]$ . It is also known as the **probability measure** of  $A$ , where 0 means impossible and 1 means certain.

A reasonable theory for modeling uncertainty must satisfy these foundational axioms, ensuring consistency and avoiding contradictions:

1. All probabilities lie between 0 and 1:  $0 \leq P(A) \leq 1$ . This bounds the measure of certainty.
2. Valid propositions (the entire sample space) have probability 1, while unsatisfiable propositions (the empty set) have probability 0:  $P(\emptyset) = 0$  and  $P(\Omega) = 1$ . This sets the scale for impossible and certain events.
3. For the union of two events (disjunction), the probability accounts for overlap:  $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ . This additivity rule prevents double-counting shared outcomes.

These axioms, proposed by Kolmogorov, form the only coherent system for probability. Notably, Bruno de Finetti (1931) showed that using them prevents an opponent from exploiting inconsistencies in gambling scenarios—any deviation would allow arbitrage.

Venn diagrams are useful for visualizing overlaps in unions and intersections. For example, imagine two overlapping circles for events  $A$  and  $B$ : The union  $A \cup B$  covers the entire area of both, while the intersection  $A \cap B$  is the overlapping region.

### Union Probability Example

Suppose  $P(A) = 0.4$ ,  $P(B) = 0.5$ , and  $P(A \cap B) = 0.2$ . Then,

$$P(A \cup B) = 0.4 + 0.5 - 0.2 = 0.7$$

This shows how to compute the probability of "A or B (or both)".

## Random Variables

### Random Variable

A **real-valued random variable**  $X$  is a function that assigns a real number to each outcome in the sample space:  $X : \Omega \rightarrow \mathbb{R}$ . It transforms outcomes into numerical values for easier analysis, like measuring quantities such as height or temperature.

### Discrete Random Variables

- Let  $X(\omega) = 1$  if a randomly drawn person  $\omega$  from the class  $\Omega$  is female, and  $X(\omega) = 0$  otherwise. This is a binary indicator variable.
- $X(\omega) =$  The hometown (encoded as a number or category) of a randomly drawn person  $\omega$  from the class  $\Omega$ .

Probabilities for random variables derive from events in the sample space. For instance,  $P(X = x)$  is the probability of the event  $\{\omega : X(\omega) = x\}$ , the set of outcomes where  $X$  takes value  $x$ .

This mapping allows us to focus on numerical properties rather than individual outcomes. Link to [Linear Algebra](#) for vector representations in ML.

## Probability Distributions

A **probability** (often lowercase  $p$ ) refers to a single numerical value for a specific outcome or event. In contrast, a **distribution** is a complete table or function specifying probabilities for all possible values of a random variable.

### Discrete Distribution for Weather

Consider a simple discrete case with Temperature ( $T$ ) and Weather ( $W$ ):

| <i>Temperature (T)</i> | <i>Probability P(T)</i> |
|------------------------|-------------------------|
| <i>Hot</i>             | <i>0.5</i>              |
| <i>Cold</i>            | <i>0.5</i>              |

| <i>Weather (W)</i> | <i>Probability P(W)</i> |
|--------------------|-------------------------|
| <i>Sunny</i>       | <i>0.6</i>              |
| <i>Cloudy</i>      | <i>0.4</i>              |

*This table shows the marginal probabilities for each variable alone.*

### ***Joint Probability Distribution***

*When two or more random variables interact, their joint distribution captures how the probability of one value depends on the others. It is denoted  $P(X = x \wedge Y = y)$ , or simply  $P(X, Y)$ , for the probability that both  $X = x$  and  $Y = y$  occur simultaneously.*

### ***Joint Distribution Table***

*To make dependencies clear, here's a full joint table:*

| <i>T \ W</i> | <i>Sunny</i> | <i>Cloudy</i> | <i>P(T)</i> |
|--------------|--------------|---------------|-------------|
| <i>Hot</i>   | <i>0.4</i>   | <i>0.1</i>    | <i>0.5</i>  |
| <i>Cold</i>  | <i>0.2</i>   | <i>0.3</i>    | <i>0.5</i>  |
| <i>P(W)</i>  | <i>0.6</i>   | <i>0.4</i>    | <i>1.0</i>  |

- *Probability that it's hot AND sunny:  $P(T = \text{hot}, W = \text{sunny}) = 0.4$ .*
- *Probability that it's hot: Marginal  $P(T = \text{hot}) = 0.4 + 0.1 = 0.5$ .*
- *Probability that it's hot OR sunny:  $P(T = \text{hot} \cup W = \text{sunny}) = P(T = \text{hot}) + P(W = \text{sunny}) - P(T = \text{hot}, W = \text{sunny}) = 0.5 + 0.6 - 0.4 = 0.7$ .*

Events in a joint distribution are often partial assignments, like the event  $P(T = \text{hot})$ , which sums the row for hot:  $0.4 + 0.1 = 0.5$ .

## Marginal Distribution

To obtain the distribution over one variable, eliminate others through **marginalization (summing out)**: Sum (or integrate) over the unwanted variables. For example,  $P(T = \text{hot}) = \sum_w P(T = \text{hot}, W = w) = 0.4 + 0.1 = 0.5$ .

**Relation between joint and marginal**: In general,  $P(X = x) = \sum_y P(X = x, Y = y)$ . This collapses the joint into a marginal by adding probabilities.

## Conditional Probability

This measures the probability of  $X$  given that  $Y$  has occurred:  $P(X | Y) = \frac{P(X, Y)}{P(Y)}$ , provided  $P(Y) > 0$ . It represents the fraction of worlds (outcomes) where  $X$  is true, restricted to those where  $Y$  is true.

**Conditional distributions**: These are full distributions over some variables, given fixed values for others. They normalize the joint over the conditioned variables.

## Conditional Table for $P(T | W)$

Using the joint table above:

| $T   W$ | <b>Sunny</b> ( $P(W = \text{sunny}) = 0.6$ ) | <b>Cloudy</b> ( $P(W = \text{cloudy}) = 0.4$ ) |
|---------|----------------------------------------------|------------------------------------------------|
| Hot     | $0.4/0.6 \approx 0.667$                      | $0.1/0.4 = 0.25$                               |
| Cold    | $0.2/0.6 \approx 0.333$                      | $0.3/0.4 = 0.75$                               |

For instance,  $P(T = \text{hot} | W = \text{sunny}) = 0.4/0.6 \approx 0.667$ , meaning given sunny weather, it's more likely to be hot.

These concepts build the foundation for reasoning about dependencies in data.

# Probabilistic Inference

## Probabilistic Inference

**Probabilistic inference** involves computing a desired probability distribution or value from known probabilities, often using conditional probabilities derived from a joint distribution. It

allows updating beliefs as new information arrives.

### ***Inference Examples***

- Compute  $P(\text{on time} \mid \text{no reported accidents}) = 0.90$ . This posterior probability reflects the agent's belief about train arrival given evidence of no accidents.
- These probabilities represent the agent's current beliefs conditioned on available evidence.
- As new evidence emerges, beliefs update accordingly:
  - $P(\text{on time} \mid \text{no accidents, 5 a.m.}) = 0.95$ , incorporating the time factor.
  - $P(\text{on time} \mid \text{no accidents, 5 a.m., raining}) = 0.80$ , now accounting for weather.

Observing new evidence systematically updates prior beliefs to posterior ones, enabling dynamic decision-making. See [Bayesian Inference](#) for advanced applications.

## **Inference by Enumeration**

One straightforward method for probabilistic inference is **inference by enumeration**, which computes probabilities by explicitly listing and summing possibilities from the full joint distribution. This is feasible for small spaces but scales poorly.

### ***Enumeration with Weather Variables***

Suppose we have a joint distribution over Winter ( $W$ : yes/no), Hot ( $H$ : yes/no), and Sunny ( $S$ : yes/no). To compute  $P(W = \text{yes})$ :

- Sum over all entries in the joint where  $W = \text{yes}$ , regardless of  $H$  and  $S$ .

For a conditional like  $P(W = \text{yes} \mid H = \text{yes}, S = \text{yes})$ :

- First, select only rows matching the evidence ( $H = \text{yes}, S = \text{yes}$ ).
- Sum the probabilities in those rows where  $W = \text{yes}$  to get the numerator.
- Normalize by the total probability of the evidence:  $P(W = \text{yes} \mid H = \text{yes}, S = \text{yes}) = \frac{P(W=\text{yes}, H=\text{yes}, S=\text{yes})}{P(H=\text{yes}, S=\text{yes})}$ .

Similarly, for  $P(W = \text{yes} \mid H = \text{yes})$ :

- Condition on  $H = \text{yes}$  (sum over  $S$ ), then normalize.

The general steps are:

1. Identify the evidence and select matching rows in the joint table.
2. For the query variable, sum the selected probabilities where it holds.
3. Divide by the total probability of the evidence (sum of selected rows).

This enumeration directly applies the definitions of conditional probability.

### *Code for Marginal by Enumeration*

*To demonstrate with code, here's a simple Python snippet to compute a marginal by enumeration:*

```
# Example: Marginal probability from joint distribution
# Joint probabilities: P(W=yes, H=yes, S=yes) = 0.1, etc. (simplified)
joint = {
    ('yes', 'yes', 'yes'): 0.1,
    ('yes', 'yes', 'no'): 0.05,
    ('yes', 'no', 'yes'): 0.05,
    ('yes', 'no', 'no'): 0.1,
    ('no', 'yes', 'yes'): 0.2,
    ('no', 'yes', 'no'): 0.1,
    ('no', 'no', 'yes'): 0.15,
    ('no', 'no', 'no'): 0.25
}

def marginal_p_w_yes(joint):
    """Compute P(W=yes) by summing over H and S"""
    return sum(prob for (w, h, s), prob in joint.items() if w == 'yes')

p_w_yes = marginal_p_w_yes(joint)
print(f"P(W=yes) = {p_w_yes}") # Output: 0.3
```

*This code iterates over the joint to sum relevant probabilities, clarifying the enumeration process.*

## Product and Chain Rule

### *Product Rule*



The **product rule** expresses joint probabilities in terms of conditionals:  $P(A, B) = P(A | B)P(B) = P(B | A)P(A)$ . It shows how to factor dependencies, making computation easier by breaking down complexes.

For multiple variables, the **chain rule** extends this: For any sequence of random variables  $X_1, \dots, X_n$ , the joint probability decomposes as  $P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i | X_1, \dots, X_{i-1})$ . The first term is simply  $P(X_1)$ , and each subsequent conditional depends only on preceding variables.

This decomposition reveals that any joint distribution can be represented as a product of conditional distributions, ordered appropriately. It underpins algorithms like **Bayesian Networks**.

### Chain Rule Example

For three variables  $A, B, C$ ,  $P(A, B, C) = P(A)P(B | A)P(C | A, B)$ . If  $A$  is temperature,  $B$  is humidity, and  $C$  is rain, this captures sequential dependencies.

The chain rule ensures full joints can always be factored, aiding inference in complex models.

## Bayes' Rule

### Bayes' Rule

**Bayes' rule** provides a way to reverse conditional probabilities:  $P(Y | X) = \frac{P(X|Y)P(Y)}{P(X)}$ , where the denominator  $P(X)$  is the marginal, computed as  $P(X) = \sum_y P(X | y)P(y)$ . This flips "cause to effect" ( $P(X | Y)$ ) into "effect to cause" ( $P(Y | X)$ ).

In Bayesian terms:

- **Posterior**  $P(Y | X) \propto$  **likelihood**  $P(X | Y) \times$  **prior**  $P(Y)$ .
- Importantly, do not confuse: The posterior is the prior times likelihood, but normalized by  $P(X)$ —omitting normalization leads to uncalibrated probabilities.

### Medical Diagnosis

In medical diagnosis, compute the probability of a disease given symptoms from the probability of symptoms given disease. Suppose  $P(\text{symptom} | \text{disease}) = 0.9$ ,  $P(\text{disease}) = 0.01$ , and  $P(\text{symptom}) = 0.05$ . Then,

$$P(\text{disease} \mid \text{symptom}) = \frac{0.9 \times 0.01}{0.05} = 0.18$$

Bayes' rule is central to updating beliefs with evidence.

## The Monty Hall Problem

The Monty Hall problem illustrates Bayes' rule in a classic probability puzzle from the game show *Let's Make a Deal*:

- There are three doors: A, B, and C, with a prize behind one (equally likely).
- The contestant picks door A initially.
- The host, knowing what's behind the doors, opens door C, revealing it empty (no prize).
- Now, the contestant can stick with A or switch to B.
- Define  $H_C$  as the event that the host opens door C.

Using Bayes' rule:

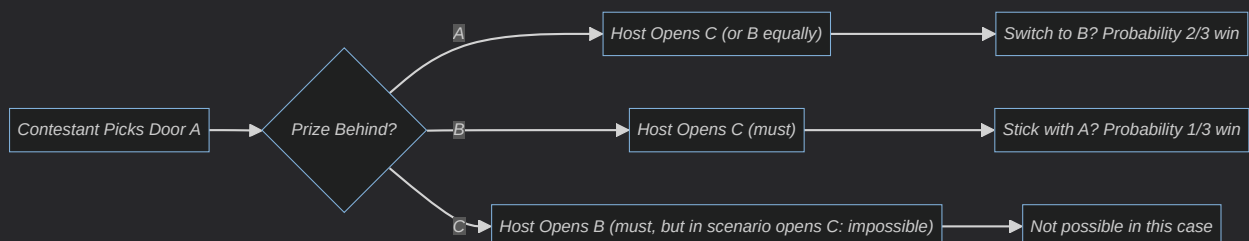
- Prior probabilities:  $P(A) = P(B) = P(C) = \frac{1}{3}$ , assuming the prize is equally likely behind any door.
- Likelihoods:  $P(H_C \mid A) = \frac{1}{2}$  (host chooses C or B equally if prize in A),  $P(H_C \mid B) = 1$  (host must open C if prize in B),  $P(H_C \mid C) = 0$  (host won't open C if prize there).
- Posterior for sticking:  $P(A \mid H_C) = \frac{P(H_C \mid A)P(A)}{P(H_C)} = \frac{(1/2)(1/3)}{1/2} = \frac{1}{3}$ .
- Posterior for switching:  $P(B \mid H_C) = \frac{P(H_C \mid B)P(B)}{P(H_C)} = \frac{1 \cdot 1/3}{1/2} = \frac{2}{3}$ .

First, compute the normalizing  $P(H_C) = P(H_C \mid A)P(A) + P(H_C \mid B)P(B) + P(H_C \mid C)P(C) = (1/2)(1/3) + 1(1/3) + 0(1/3) = 1/2$ . Thus,  $P(B \mid H_C) = \frac{1/3}{1/2} = \frac{2}{3}$ , and  $P(A \mid H_C) = \frac{1/6}{1/2} = \frac{1}{3}$ .

Switching wins with probability  $\frac{2}{3}$ , as the host's action concentrates probability on the unchosen door.

### Monty Hall Decision Flowchart

To visualize the decision process, consider this flowchart:



*This diagram shows how the host's reveal shifts probabilities, clarifying why switching is advantageous.*

## Independence and Conditional Independence

### *Independence*

**Independence:** Two random variables  $X$  and  $Y$  are independent if their joint probability factors completely:  $P(X, Y) = P(X)P(Y)$ . Equivalently,  $P(X | Y) = P(X)$ , meaning observing  $Y$  provides no information about  $X$ —the variables do not influence each other.

### *Independence Examples*

- *Independent:* The outcome of winning on roulette this week and next week, as spins are unrelated.
- *Dependent:* Shots in Russian roulette, where previous survivals affect the next due to the shared gun state.

Independence simplifies computations, as joints become products of marginals.

### *Conditional Independence*

**Conditional independence:**  $X$  is conditionally independent of  $Y$  given  $Z$ , denoted  $X \perp Y | Z$ , if  $P(X | Y, Z) = P(X | Z)$ . Here,  $Y$  adds no extra information about  $X$  once  $Z$  is known.

### *Toothache, Cavity, Catch*

Consider variables: Toothache (+ or -), Cavity (+ or -), Catch (+ or -).

- $P(+catch | +toothache, +cavity) = P(+catch | +cavity)$ , as toothache doesn't affect catch given cavity.
- $P(+catch | +toothache, -cavity) = P(+catch | -cavity)$ .
- Thus,  $Catch \perp Toothache | Cavity$ :  $P(Catch | Toothache, Cavity) = P(Catch | Cavity)$ .

- This is equivalent to  $P(\text{Toothache} \mid \text{Catch}, \text{Cavity}) = P(\text{Toothache} \mid \text{Cavity})$ , or the joint factors:  $P(\text{Toothache}, \text{Catch} \mid \text{Cavity}) = P(\text{Toothache} \mid \text{Cavity})P(\text{Catch} \mid \text{Cavity})$ .

### London Taxi Drivers

There's a correlation between accidents and wearing coats (coats might seem to "cause" accidents), but conditionally independent given rain—coats are worn only when raining, so rain explains the link.

Conditional independence is key in models like Naïve Bayes, reducing complexity.

## Model-Based Classification with Naïve Bayes

In machine learning, models are learned from data to predict outputs:  $y = f(x)$ , where parameters (like probabilities) are estimated from training examples. See [Machine Learning Algorithms](#) for more classifiers.

### Naïve Bayes

**Naïve Bayes:** This is a generative model for classification that assumes all attributes (features) are independent given the class label  $y$ . Despite the "naïve" independence assumption, it often performs well.

- Estimating the class prior  $P(y)$  is straightforward from data frequencies.
- The likelihood  $P(x \mid y)$  is challenging for high-dimensional  $x$ , but the independence assumption simplifies it:  $P(x \mid y) = \prod_i P(x_i \mid y)$ , where  $x_i$  are individual features.
- Without the assumption, estimating  $P(x \mid y)$  requires  $\sim 2^{dK}$  parameters ( $d$  attributes,  $K$  classes, binary features); with it, only  $\sim 2^d K$ , a huge saving.

**Naïve Bayes classifier:** Applies Bayes' rule:  $P(y \mid x) = \frac{P(y)P(x|y)}{P(x)} = \frac{P(y) \prod_i P(x_i|y)}{P(x)}$ . Since  $P(x)$  is constant for classification, we use  $\arg \max_y P(y) \prod_i P(x_i \mid y)$ .

### Play Tennis?

Features: Outlook (sunny/overcast/rain), Temperature (hot/mild/cool), Humidity (high/normal), Wind (strong/weak).

Classes: Yes/No (play tennis).

Sample tables from training data (14 examples 9 Yes, 5 No):

*Prior  $P(y)$ :*

| <i>Class</i> | <i><math>P(y)</math></i> |
|--------------|--------------------------|
| Yes          | $9/14 \approx 0.64$      |
| No           | $5/14 \approx 0.36$      |

***Likelihood**  $P(\text{feature} \mid y)$  (counts + 1 for Laplace smoothing, see below):*

*For Outlook given Yes: Sunny 2/9, Overcast 4/9, Rain 3/9.*

*Predictions: For input (sunny, hot, high, strong):*

- $P(\text{Yes} \mid x) \propto 0.64 \times (2/9) \times (2/9) \times (3/9) \times (3/9) \approx 0.0053$ .
- $P(\text{No} \mid x) \propto 0.36 \times (3/5) \times (2/5) \times (4/5) \times (3/5) \approx 0.0206$ .
- *Normalize: No has higher posterior (0.795 vs. 0.205), so classify as No.*

*Classify by  $\arg \max_y P(y \mid x)$ .*

### ***Naïve Bayes Summary***

- *Computationally efficient: Training is one pass over data to count frequencies; classification is linear in the number of features.*
- *Performs well empirically, even if independence doesn't hold perfectly.*
- *Ideal for moderate to large training sets with many attributes, like text classification (e.g., spam detection).*

### ***Naïve Bayes Code***

*To implement a simple Naïve Bayes prediction in code:*

```
# Example: Naïve Bayes classification (simplified, no smoothing)
# Priors
p_yes = 0.64
p_no = 0.36

# Likelihoods for features: [P(sunny|class), P(hot|class), P(high|c
lik_yes = [2/9, 2/9, 3/9, 3/9]
lik_no = [3/5, 2/5, 4/5, 3/5]
```

```

x = [1, 1, 1, 1] # Binary: 1 if matches (sunny, hot, high, strong)

# Unnormalized posteriors
unnorm_yes = p_yes
unnorm_no = p_no
for i in range(4):
    unnorm_yes *= lik_yes[i] if x[i] else (1 - lik_yes[i])
    unnorm_no *= lik_no[i] if x[i] else (1 - lik_no[i])

total = unnorm_yes + unnorm_no
p_yes_given_x = unnorm_yes / total
p_no_given_x = unnorm_no / total

print(f"P(Yes|x) ≈ {p_yes_given_x:.3f}, P(No|x) ≈ {p_no_given_x:.3f}")
# Classify as No

```

*This snippet computes the posteriors, demonstrating the product's efficiency.*

## Laplace Smoothing

A common issue in Naïve Bayes: If an attribute value never appears for a class in training,  $P(x_i | y) = 0$ , causing the entire product to be zero and breaking classification.

### *Laplace Smoothing*

**Laplace smoothing** (add-one smoothing) addresses this by adding pseudocounts:  $P(x_i | y) = \frac{\text{count}(x_i, y) + 1}{\text{count}(y) + |\text{values}|}$ , where  $|\text{values}|$  is the number of possible values for  $x_i$ .

### *Coin Flip Smoothing*

For a coin flip (Bernoulli), if 3 heads in 3 flips (no tails), unsmoothed  $P(\text{tail}) = 0$ . With Laplace:  $P(\text{tail}) = \frac{0+1}{3+2} = 0.25$ . For features, if "strong" wind never seen for Yes (0/9), smoothed  $P(\text{strong} | \text{Yes}) = \frac{0+1}{9+2} \approx 0.091$ .

This prevents zero probabilities and improves robustness to sparse data.

## Discriminative vs. Generative Learning

Supervised learning aims to estimate the conditional  $P(Y | X)$  from labeled data  $(X, Y)$ .

- **Generative learning:** Models the full joint  $P(X, Y) = P(X | Y)P(Y)$ , then infers  $P(Y | X)$  via Bayes. Naïve Bayes is generative, learning how data is generated from classes.
- **Discriminative learning:** Directly estimates  $P(Y | X)$  without modeling  $P(X)$ , focusing on the decision boundary. **Logistic Regression** is discriminative, optimizing for classification accuracy.

Generative models are useful when data generation is of interest; discriminative are often more accurate for prediction alone.

## Maximum a Posteriori (MAP) & Bayesian Learning

### *Hypothesis*

A ***hypothesis***  $h$  is a probabilistic theory describing the domain, such as parameters of a distribution.

### *Bayesian Learning*

**Bayesian learning** treats hypotheses as random variables and updates their probabilities with data  $d$ :  $P(h | d) = \frac{P(d|h)P(h)}{P(d)}$ , where  $P(d) = \sum_h P(d | h)P(h)$ .

For predictions on unknown  $X$ , average over hypotheses:  $P(X | d) = \sum_h P(X | h)P(h | d)$ . This weights predictions by posterior belief in each  $h$ .

- The data  $d$  consists of observed examples  $(x_j, y_j)$ .
- In practice, the **Maximum a Posteriori (MAP)** hypothesis  $h_{MAP} = \arg \max_h P(h | d)$  approximates full Bayesian by using the best hypothesis alone, valid if one  $h$  dominates.

When conditional independence holds, Naïve Bayes classification equates to MAP over class hypotheses.

### *i.i.d. Assumption*

#### ***i.i.d. (Independent and Identically Distributed):***

Learning assumes training data are i.i.d.: Each example  $e_j = (x_j, y_j)$  is independently drawn from a fixed underlying distribution  $P(X, Y)$ .

- **Stationary assumption:** The distribution doesn't change over time.
- Independence ensures examples don't influence each other.

- *Identically distributed connects past observations to future predictions; without it, the future would be unpredictable.*

*This i.i.d. assumption justifies using empirical frequencies to estimate probabilities.*

**MAP** → **ML**: If the prior  $P(h)$  is uniform (equal for all  $h$ ), MAP reduces to maximizing the likelihood  $P(d | h)$ , known as **Maximum Likelihood (ML)** estimation.

- *Priors penalize complex hypotheses (e.g., via regularization).*
- *For large data,  $MAP \approx ML$ , as data overwhelms the prior.*

## Estimating Probabilities: Maximum Likelihood Estimation (MLE)

For data generated from a parametric model  $P(\text{data} | \theta)$  with unknown parameters  $\theta$ , **MLE** finds  $\hat{\theta}_{ML} = \arg \max_{\theta} P(\text{data} | \theta)$ , often via log-likelihood for tractability.

### Coin Flips MLE

*For  $n$  flips with  $h$  heads,  $P(\text{data} | \theta) = \theta^h (1 - \theta)^{n-h}$ . The MLE is  $\hat{\theta} = \frac{h}{n}$ , the observed frequency.*

*Steps for general MLE:*

1. *Write the likelihood  $L(\theta) = P(\text{data} | \theta)$ .*
2. *Take the log:  $\ell(\theta) = \log L(\theta)$ , summing logs for products.*
3. *Compute the derivative  $\frac{d}{d\theta} \ell(\theta)$  and set to zero to solve for  $\hat{\theta}$ .*

*This maximizes the probability of observing the data under the model.*

## Discrete Probability Distributions

### Bernoulli Distribution

**Bernoulli Distribution** --  $\text{Ber}(\theta)$ : Models a single binary trial (e.g., coin toss), with sample space  $\Omega = \{\text{head}, \text{tail}\}$ ,  $P(\text{head}) = \theta$ ,  $P(\text{tail}) = 1 - \theta$ .

### Bernoulli Example

*For  $\theta = 0.6$ ,  $P(\text{head}) = 0.6$ . Numerical: One flip yielding head has probability 0.6.*

### Binomial Distribution



**Binomial Distribution** --  $\text{Bin}(n, \theta)$ : Extends to  $n$  independent Bernoulli trials, counting  $k$  successes (heads):  $P(K = k) = \binom{n}{k} \theta^k (1 - \theta)^{n-k}$ . The sample space  $\Omega$  has  $2^n$  sequences, but we observe the count  $K \in \{0, 1, \dots, n\}$ .

### Binomial Example

For  $n = 3, \theta = 0.5, P(K = 2) = \binom{3}{2} (0.5)^2 (0.5)^1 = 3 \times 0.125 = 0.375$ .

## MAP for Coin Flip

For binomial likelihood, a **Beta prior**  $\text{Beta}(\alpha, \beta)$  is conjugate (posterior same family): Prior  $p(\theta) \propto \theta^{\alpha-1} (1 - \theta)^{\beta-1}$ .

- Posterior:  $\text{Beta}(\alpha + h, \beta + (n - h))$ , equivalent to observing  $\alpha - 1$  extra heads and  $\beta - 1$  tails.
- MAP estimate: Mode of posterior,  $\hat{\theta}_{MAP} = \frac{\alpha + h - 1}{\alpha + \beta + n - 2}$ .
- For small  $n$ , prior influences heavily; as  $n$  grows, posterior mean approaches MLE, and prior is "forgotten."

### Beta Prior Example

Uniform prior  $\text{Beta}(1, 1)$ , after 3 heads: Posterior  $\text{Beta}(4, 1)$ ,  $\text{MAP} = 3/4 = 0.75$  (vs.  $\text{MLE} = 1$ ).

## Continuous Random Variables

### Continuous Random Variable

A random variable  $X$  is **continuous** if its possible values form an interval on the real line, and the probability of any exact value is zero:  $P(X = x) = 0$ . Instead, probabilities are over intervals.

The **Probability Density Function (PDF)**  $f(x)$  satisfies  $P(a < X < b) = \int_a^b f(x) dx$ , with  $f(x) \geq 0$  and  $\int_{-\infty}^{\infty} f(x) dx = 1$ . For small  $dx$ ,  $P(x < X < x + dx) \approx f(x) dx$ .

The **Cumulative Distribution Function (CDF)**  $F(x) = P(X \leq x) = \int_{-\infty}^x f(t) dt$ , where  $f(x) = \frac{d}{dx} F(x)$ .

### Continuous Distributions

- **Uniform Distribution** on  $[a, b]$ :  $f(x) = \frac{1}{b-a}$  for  $x \in [a, b]$ , else 0. Constant density over interval.
  - Numerical:  $\text{Uniform}[0, 1]$ ,  $P(0.2 < X < 0.5) = 0.3$ .
- **Normal (Gaussian) Distribution**  $N(\mu, \sigma^2)$ :  $f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$ . Bell-shaped, symmetric around mean  $\mu$ , spread by  $\sigma$ .
  - Numerical:  $N(0, 1)$ ,  $f(0) \approx 0.399$ .

Continuous distributions model real-world measurements like heights or errors. Link to [Neural Networks](#) for Gaussian assumptions in loss functions.

## Moments

### Moments

**Moments** summarize key characteristics of a distribution's shape and location.

- **Mean (expectation)**  $\mu = E[X]$ : The average value,  $\mu = \int_{-\infty}^{\infty} x f(x) dx$  for continuous  $X$ , or  $\mu = \sum_x x p(x)$  for discrete (where  $p(x)$  is probability mass or density).
  - **Example:** For  $\text{Uniform}[0, 1]$ ,  $E[X] = \int_0^1 x \cdot 1 dx = 0.5$ .
- **Variance**  $\text{Var}(X) = E[(X - \mu)^2] = E[X^2] - (E[X])^2$ : Measures spread around the mean.
  - **Example:** For  $\text{Uniform}[0, 1]$ ,  $\text{Var}(X) = E[X^2] - (0.5)^2 = \int_0^1 x^2 dx - 0.25 = \frac{1}{3} - 0.25 = \frac{1}{12} \approx 0.083$ .

Higher moments include skewness (asymmetry) and kurtosis (tailedness), but mean and variance are foundational.

## MLE for Gaussian Mean and Variance

For i.i.d. samples  $x_1, \dots, x_n \sim N(\mu, \sigma^2)$ , the likelihood is  $L(\mu, \sigma^2) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i-\mu)^2}{2\sigma^2}\right)$ .

Log-likelihood:  $\ell(\mu, \sigma^2) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum (x_i - \mu)^2$ .

- For  $\mu$  (fix  $\sigma^2$ ): Derivative w.r.t.  $\mu$  sets  $\hat{\mu}_{ML} = \frac{1}{n} \sum x_i$ , the sample mean (unbiased estimator).
- For  $\sigma^2$  (fix  $\mu$ ):  $\hat{\sigma}_{ML}^2 = \frac{1}{n} \sum (x_i - \hat{\mu})^2$ , the average squared deviation (biased, underestimates).
- Unbiased variance:  $\hat{\sigma}^2 = \frac{1}{n-1} \sum (x_i - \hat{\mu})^2$ , dividing by  $n - 1$  to correct for sample size.

These derive from maximizing the log-likelihood, central to fitting Gaussian models in ML.

### Gaussian MLE Example

Data  $\{1, 3, 2\}$ ,  $n = 3$ .  $\hat{\mu} = (1 + 3 + 2)/3 = 2$ . ML variance =  $\frac{(1-2)^2 + (3-2)^2 + (2-2)^2}{3} = \frac{2}{3} \approx 0.667$ ; unbiased =  $2/2 = 1$ .

## Exercise on MLE and MAP

### Poisson Dataset Exercise

**Dataset**  $D = \{2, 5, 9, 5, 4, 8\}$ , assumed i.i.d. from  $\text{Poisson}(\lambda)$ , where Poisson PMF is  $P(X = k \mid \lambda) = \frac{\lambda^k e^{-\lambda}}{k!}$ .

**MLE:**

- Likelihood:  $L(\lambda) = \prod_i \frac{\lambda^{x_i} e^{-\lambda}}{x_i!} = \frac{\lambda^{\sum x_i} e^{-n\lambda}}{\prod x_i!}$ , with  $\sum x_i = 33$ ,  $n = 6$ .
- Log-likelihood:  $\ell(\lambda) = (\sum x_i) \log \lambda - n\lambda - \sum \log(x_i!)$ .
- Derivative:  $\frac{d}{d\lambda} \ell = \frac{\sum x_i}{\lambda} - n = 0 \implies \hat{\lambda}_{ML} = \frac{\sum x_i}{n} = \frac{33}{6} = 5.5$ .

To compute in code:

```
import math
from scipy.stats import poisson

# Data
data = [2, 5, 9, 5, 4, 8]
n = len(data)
sum_x = sum(data)

# MLE: lambda_hat = mean
lambda_ml = sum_x / n
print(f"MLE λ = {lambda_ml}") # 5.5

# Verify log-likelihood at MLE (ignoring constants)
log_lik = sum_x * math.log(lambda_ml) - n * lambda_ml
print(f"Log-likelihood at MLE: {log_lik}") # Approximate value
```

**MAP** with Gamma prior  $\Gamma(\lambda \mid k = 3, \theta = 1)$ : Prior  $p(\lambda) \propto \lambda^{k-1} e^{-\lambda/\theta} = \lambda^2 e^{-\lambda}$ .

- $\text{Posterior} \propto L(\lambda) \times \text{prior} \propto \lambda^{33} e^{-6\lambda} \times \lambda^2 e^{-\lambda} = \lambda^{35} e^{-7\lambda}.$
- This is  $\text{Gamma}(36, 1/7)$ , mode (MAP)  $\hat{\lambda}_{MAP} = \frac{36-1}{7} = 5.$
- General formula:  $\hat{\lambda}_{MAP} = \frac{\sum x_i + k - 1}{n + 1/\theta} = \frac{33 + 3 - 1}{6 + 1} = \frac{35}{7} = 5.$
- For large  $n$ ,  $\hat{\lambda}_{MAP} \approx \hat{\lambda}_{ML}$ , as data dominates the prior.

In this case, MAP pulls toward the prior mean  $k\theta = 3$ , yielding 5 vs. MLE 5.5.

## Frequentist vs. Bayesian: MLE vs. MAP

- **Frequentist (MLE):** Views  $\theta$  as fixed but unknown, data as random. Estimates  $\hat{\theta}$  to maximize data likelihood. Reliable for large samples, but ignores prior knowledge.
- **Bayesian (MAP):** Treats  $\theta$  as random with a prior  $P(\theta)$ . MAP maximizes posterior, incorporating beliefs. Better for small data (prior regularizes), but choice of prior affects results.

MLE is unbiased for large data; MAP introduces bias but reduces variance via prior penalties on complexity.

## How Good is the Estimator? Bias and Variance

Evaluating estimators involves **bias** and **variance**, trading off accuracy and stability.

### Bias

**Bias:**  $B(\hat{\theta}) = E[\hat{\theta}] - \theta$ , the systematic error. An estimator is unbiased if  $E[\hat{\theta}] = \theta$  on average over datasets.

### Bias Examples

- Bernoulli mean  $\hat{\theta} = \frac{1}{m} \sum_{i=1}^m x^{(i)}$ :  $E[\hat{\theta}] = \theta$ , unbiased.
- Gaussian mean  $\bar{x} = \frac{1}{n} \sum x_i$ : Unbiased,  $E[\bar{x}] = \mu$ .
- Gaussian variance  $\frac{1}{n} \sum (x_i - \bar{x})^2$ : Biased low,  $E[\hat{\sigma}^2] = \frac{n-1}{n} \sigma^2 < \sigma^2$ ; unbiased version uses  $n - 1$ .

### Variance

**Variance:**  $\text{Var}(\hat{\theta}) = E[(\hat{\theta} - E[\hat{\theta}])^2]$ , measuring how much  $\hat{\theta}$  fluctuates across different training sets (treating the dataset as random).

- The **standard error**  $SE(\hat{\theta}) = \sqrt{\text{Var}(\hat{\theta})}$  quantifies precision.
- For the sample mean:  $SE(\bar{x}) = \frac{\sigma}{\sqrt{n}}$ , estimated by replacing  $\sigma$  with sample standard deviation.

Among unbiased estimators, the one with lowest variance is preferred (e.g., via Cramer-Rao bound).

**Mean Squared Error (MSE):**  $E[(\hat{\theta} - \theta)^2] = B(\hat{\theta})^2 + \text{Var}(\hat{\theta})$ , combining squared bias and variance. Lower MSE is better overall.

**Bias-Variance Decomposition:** Total expected error for predictions decomposes as  $\text{bias}^2 + \text{variance} + \text{irreducible error (noise)}$ . High bias underfits; high variance overfits. Good estimators balance both.

## Example: Rate of a Poisson Process

Suppose users access a webserver following a  $\text{Poisson}(\lambda)$  process per hour. Data from 4 periods: counts  $\{3, 4, 2, 5\}$  (sum=14,  $n = 4$ , mean=3.5).

Two estimators:

1. MLE  $\hat{\lambda}_1 = \frac{1}{4} \sum \text{counts} = 3.5$ , unbiased ( $E[\hat{\lambda}_1] = \lambda$ ).
  2. Smoothed  $\hat{\lambda}_2 = 0.8 \times 3.5 + 0.2 \times 2 = 3.2$  (shrink toward prior guess of 2), biased low but lower variance.
- Bias:  $\hat{\lambda}_1$  has 0;  $\hat{\lambda}_2$  has  $E[\hat{\lambda}_2] = 0.8\lambda + 0.4 < \lambda$ .
  - Variance:  $\hat{\lambda}_1 = \lambda/4 \approx \lambda/4$ ;  $\hat{\lambda}_2 = 0.8^2 \times \lambda/4 < \text{Var}(\hat{\lambda}_1)$ .
  - MSE: For small true  $\lambda$  (e.g., 1), biased  $\hat{\lambda}_2$  has lower MSE; for large  $\lambda$  (e.g., 10), unbiased  $\hat{\lambda}_1$  is better.

Choice depends on expected  $\lambda$  and risk tolerance—e.g., conservative estimates for low rates.

### Poisson MSE Simulation Code

To compute MSE in code for simulation:

```
import numpy as np

def poisson_mse(lambda_true, n=4, num_sims=10000):
    """Simulate MSE for MLE Poisson rate"""
    estimates = []
    for _ in range(num_sims):
```

```
data = np.random.poisson(lambda_true, n)
lambda_hat = np.mean(data)
estimates.append(lambda_hat)
mse = np.mean((np.array(estimates) - lambda_true)**2)
return mse

print(f"MSE for  $\lambda=1$ : {poisson_mse(1):.3f}") # ~1.000 (var=1/n=0.25)
print(f"MSE for  $\lambda=10$ : {poisson_mse(10):.3f}") # ~2.500 (var=10/4=2.5)
```

*This simulates datasets to empirically compute MSE, showing variance scales with  $\lambda$ .*

## Summary

This module covers:

- 
- Basic Probability.
  - Bayes' Rule.
  - Naïve Bayes classifier.
  - Discriminative vs. Generative Learning.
  - Maximum a Posteriori.
  - Maximum Likelihood.
  - i.i.d. data.
  - Discrete and Continuous Distributions.
  - Bias / Variance.
  - Mean Squared Error.

Expected values and variance provide the statistical backbone for machine learning, enabling robust modeling under uncertainty. These concepts integrate into advanced techniques like [Neural Networks](#) and ensemble methods.

## References

- [Machine Learning](#)
- [Bayesian Inference](#)
- [Linear Algebra](#)
- [Logistic Regression](#)
- [Bayesian Networks](#)

